

1
2
3

4
5

NOCK & THE ART OF
SOLID-STATE COMPUTING
An Enquiry into the Virtual

N. E. Davis
~lagrev-nocfep

6 Contents

7	1 The Roots of Computing in the Ancient World	1
8	1.1 Tabulation	2
9	1.2 Solid-state indices	2
10	2 Combinators for Fun and Profit	3
11	2.1 Combinators in Computing	3
12	2.1.1 The SKI calculus	4
13	2.1.2 The BCKW calculus	5
14	2.2 Nock Opcodes as Combinator	6
15	2.2.1 I Identity = Opcode 0	8
16	2.2.2 K Constant = Opcode 1	8
17	2.2.3 S Substitution = Opcode 2	8
18	2.2.4 The Axiomatic Operators	9
19	2.3 Idioms & Design Patterns	10
20	2.3.1 C Flip Operator	10
21	2.4 Nock as Generally Computable	10
22	2.5 Virtualization	11
23	3 Interpreter Runtime	13

Chapter 1

The Roots of Computing in the Ancient World

Abstract

Cuneiform and early computational concepts are explored in this chapter.

Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature.

(Keiji Nishitani, *Religion and Nothingness*, 1982, pp. 129–130)

There are many worthy raconteurs of the history of computing, such as **Hofstadter1980** (**Hofstadter1980**) and **Penrose2004** (**Penrose2004**). What distinguishes my approach to the history of computing from other tellings? Computing has deep connexions to alchemy and true language, which I hope to demonstrate without assuming any Hermetic background on the reader's part. Much like technology, computing is not merely a tool but a stance towards the world. However, it does not entail technologism's supercession; computing is always both a lens and a kaleidoscope, through which many layers of reality can be seen simultaneously. It is a way of seeing the world as a system of pure statements and acting to make those statements true. It is both ontology and epistemology.

First, therefore, I must establish that computation is the apex of a long human search for a language of truth. The origin of symbol itself in the σύμβολον, a

48 token broken in half to infallibly mark identity, is one font. Later “symbol” was
49 the profession of faith, both an attempt to formulate truth and a sign of shared
50 claims. The primary source, however, is language itself, Hofstadter’s “strange
51 loop” of self-reference. The first possible Nock formula is a crash, the void. The
52 second is self-reference, awakening. Peirce would be proud of the progression.
53 We have tricked ourselves into thinking that computing is a late development,
54 a fruitful elaboration of the end of mathematics. In fact, it is the philosopher’s
55 stone, and it underlies every process in the world. It has turned lead into gold.

56 The historical account which follows aims throughout at the goal of under-
57 standing what Nock is and why it matters as a protocol. The reader will under-
58 stand what mankind has been looking for, when we have caught various pieces
59 of the puzzle, how these things came together in the mid-20th century, and how
60 they have been refined into a pure machine.

61 Outline: - Cuneiform, tabulation, calculation, and algorithm - Solid-state in-
62 dices: abaci, tokens, etc. Mechanism as embodied equation. - Pure language and
63 true statements (Lull, Leibniz, Wilkins) - Generalization (Jacquard, Babbage) -
64 The universal machine (Turing, Gödel, Zuse, von Neumann) - Pure statements as
65 Pages from the Book (Curry, Church)

66 1.1 Tabulation

67 1.2 Solid-state indices

68 The problem is that any calculation lives in someone’s head or someone’s claim.
69 This is a summary, but not the proof or solution.

70 Chapter 2

71 Combinators for Fun and Profit

72 Abstract

73 The first section of this chapter will introduce the basic SKI and BCKW combi-
74 nator calculi and examine how they can represent each other. The second section
75 will introduce Nock’s opcodes as combinators, and show how they can be used to
76 express various combinator idioms. The third section will show how to build prac-
77 tical affordances for common programming tasks, such as conditionals, recursion,
78 subprocedure invocation, and data structures. The final section will discuss the
79 relationship between combinators and virtualization, and how Nock can be used
80 to build a virtual machine even in a crash-only context.

81 2.1 Combinators in Computing

82 The pioneers of computing elaborated three complementary visions:

- 83 1. The Turing machine, a simple abstract machine that can compute anything
84 computable.
- 85 2. The lambda calculus, a pure functional language based on substitution and
86 abstraction.
- 87 3. Combinatory logic, a variable-free functional language based on combina-
88 tors.

89 Logicians early demonstrated the equivalence of these approaches, but par-
90 ticular formulations are still preferred for particular tasks. The Turing machine is
91 the most intuitive model of computation, and is often used to prove computabil-
92 ity results. The lambda calculus is the basis of many functional programming

languages, such as Lisp, Scheme, and Haskell. Combinatory logic is less widely used in practice, but has a long history in mathematical logic and type theory. It is also the basis of Nock.

Before we get too far out over our skis, let's talk about what a combinator is. At its simplest, a combinator is a function that takes one or more arguments and returns a result, without referring to any variables outside its own arguments. In practice, we'll use different combinators to build up more complex functions, and we'll see informally how they can be combined to express any computable function. We can even build combinators out of other combinators. The objective of combinatory logic is to express all computation in terms of combinators alone, without the need for variables or named functions.

This first section will introduce some elemental and compound combinators and make the case that they circumscribe "computing" as a concept.

2.1.1 The SKI calculus

Computable functions can be expressed using minimal sets of combinators. Many logicians today prefer to work with Moses Schönfinkel and Haskell Curry's SKI combinator calculus, which has three combinators:

The identity combinator I is conventionally written $Ix = x$. This simply means that it returns its single argument unchanged. There is little to say of this combinator at this point, but logicians have found that it frequently simplifies expressions.

$$I\ x ==\ x$$

The constant combinator K is conventionally written $Kxy = x$. This means that it takes two arguments and returns the first, ignoring the second. It is useful for building functions that always return a fixed value.

Questions

1. What is the result of $I\ x$?
2. What is the result of $I\ I\ x$?

$$K\ x\ y ==\ x$$

The substitution combinator S is conventionally written $Sxyz = xz(yz)$. This means that it takes three arguments: a function x , and two values y and z . It applies the function x to the value z , and also applies the function y to the value z , then applies the result of $x\ z$ to the result of $y\ z$. This combinator is more

complex than the others, but it is very powerful, as it allows us to express function application and composition.

$$S\ x\ y\ z == x\ z\ (y\ z)$$

The SKI system is Turing complete ($= \mu$ -recursive), meaning that any computable function can be expressed using only these three combinators. It is also equivalent to the lambda calculus, meaning that any lambda expression can be translated into an equivalent SKI expression and vice versa (Kleene (1952), Theorem XXX).¹

Combinators can be combined to form more complex expressions. For example, the combinator $S\ K\ K$ is equivalent to the identity combinator I , which can be proven thus:

$$\begin{aligned} S\ K\ K\ z &== && ;\ x \rightarrow K,\ y \rightarrow K,\ z \rightarrow z \\ K\ z\ (K\ z) &== && ;\ K\ z \rightarrow z \\ z & && \end{aligned}$$

In such cases, combinators are applied from left to right by the first possible reduction. This means that they will bottom out in a particular expression, which may be the argument (as with $I\ x == x$).

2.1.2 The BCKW calculus

The BCKW combinator calculus is an alternative to SKI, with a different set of primitive combinators. It has four combinators:

The composition combinator B is conventionally written $Bxyz = x(yz)$. This means that it takes three arguments: a function x , and two values y and z . It applies the function y to the value z , then applies the function x to the result.

$$B\ x\ y\ z == x\ (y\ z)$$

The flip combinator C is conventionally written $Cxyz = xzy$. This means that it takes three arguments: a function x , and two values y and z . It applies the

¹Four approaches to general-purpose computing were devised around the same time: Moses Schönfinkel and Haskell Curry's combinator SKI calculus (1920s); Alonzo Church's lambda calculus λC (1936); Alan Turing's tape machine (1936); and Stephen Kleene's μ -recursive functions (1936) building on Kurt Gödel's primitive recursive (1931) and general recursive functions (1934). These have been demonstrated to be equivalent to each other by means of the Church–Turing thesis: “Every effectively calculable function is a computable function.” Various auxiliary proofs establish the isomorphic nature of these systematic approaches; however, a footnote much larger than the length of this book is warranted to address the relationships amongst these four formulations. We simply take for granted their equivalence and move freely between representations as necessary. Furthermore, other formulations, such as the structured program theorem for control flow graphs (Böhm and Jacopini, 1966), exist and merit future exploration.

160 function x to the values z and y , effectively reversing the order of the last two
 161 arguments.

162
 163 $C\ x\ y\ z == x\ z\ y$
 164

165 The join combinator $\mathbb{J}$ is conventionally written $\mathbb{J}xy = xyy$. This means that
 166 it takes two arguments: a function x and a value y . It applies the function x to the
 167 value y twice.

168
 169 $\mathbb{J}\ x\ y == x\ y\ y$
 170

171 The SKI and BCKW systems are equivalent, meaning that any expression in
 172 one system can be translated into an equivalent expression in the other system.
 173 For example, the identity combinator I can be expressed in BCKW as:

174 $I = \mathbb{J}\ K$
 175 $\quad U$
 176 $\quad Y$
 177 $Y = S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))$

178 $\mathbb{J}$ ABT or whatever it is

179 Chris Barker The iota combinator $\imath$ can produce all the others, but in convo-
 180 luted ways.

181 Combinator calculi are generally considered too minimalist for practical pro-
 182 gramming; they shine, however, in mathematical proofs. This means that we can
 183 use them to define a standard of behavior for an algorithm, but implement the
 184 logic in a compliant form using whatever computational tools we like. Thus it is
 185 better to think of Nock not as a bare programming language, but as a standard of
 186 computation which can under many circumstances be used as the computation
 187 itself.²

188 2.2 Nock Opcodes as Combinator

189 The Nock specification is produced in full in Listing 2.1.

Listing 2.1: Nock 4K

190 Nock 4K
 191
 192 A noun is an atom or a cell. An atom is a natural number. A cell is an ordered pair of nouns.
 193
 194 Reduce by the first matching pattern; variables match any noun.

²Consideration of when to do so is largely driven by performance bottlenecks.

```

195
196 nock(a)          *a
197  [a b c]         [a [b c]]
198
199  ?[a b]           0
200  ?a               1
201  +[a b]           +[a b]
202  +a               1 + a
203  =[a a]           0
204  =[a b]           1
205
206  /[1 a]           a
207  /[2 a b]         a
208  /[3 a b]         b
209  /[(a + a) b]     /[2 /[a b]]
210  /[(a + a + 1) b] /[3 /[a b]]
211  /a               /a
212
213  #[1 a b]         a
214  #[(a + a) b c]   #[a [b /[(a + a + 1) c]] c]
215  #[(a + a + 1) b c] #[a [/[(a + a) c] b] c]
216  #a               #a
217
218  *[a [b c] d]     *[a b c] *[a d]]
219
220  *[a 0 b]         /[b a]
221  *[a 1 b]         b
222  *[a 2 b c]       *[*[a b] *[a c]]
223  *[a 3 b]         ?*[a b]
224  *[a 4 b]         +*[a b]
225  *[a 5 b c]       =[*[a b] *[a c]]
226
227  *[a 6 b c d]     *[a *[[c d] 0 *[[2 3] 0 *[a 4 4 b]]]]
228  *[a 7 b c]       *[*[a b] c]
229  *[a 8 b c]       *[[*[a b] a] c]
230  *[a 9 b c]       *[*[a c] 2 [0 1] 0 b]
231  *[a 10 [b c] d]  #[b *[a c] *[a d]]
232
233  *[a 11 [b c] d]  *[[*[a c] *[a d]] 0 3]
234  *[a 11 b c]     *[a c]
235
236  *a               *a

```

237 Nock has the following characteristics:
238 (from handout)

2.2.1 I Identity = Opcode 0

Nock’s binary tree addressing is fully general and yields the identity combinator I as a special case. (The cost, of course, is dragging in some machinery for basic arithmetic, as we will see—this is unnecessary in the base case but needed to describe and navigate nouns.)

Conceptually, we could write this in a hybrid form between combinators and Nock thus:

$$*[x\ I] == *[x\ 0\ 1] == /[1\ x] == x$$

In a sense, I resolves immediately to $[0\ 1]$, which is true but highlights a difference in terminology and argument structure. Nock conceives of itself as applying a formula to a subject, which in combinator terms means the subject is one argument and the formula(s) is/are another. However, the “position” of each will not be consistent, so you cannot simply map x to “formula”.

Nock’s homoiconicity, or the unification in representing code and data, renders it distinct in some ways. The slot operator $/$ is more general than combinators typically are, as it assumes a structure to the input which is elided in combinatory logic. This is a key difference between Nock and combinatory logic: Nock is a language for manipulating structured data, while combinatory logic is a language for manipulating functions. The slot operator $/$ is a way to access parts of structured data, which is not typically part of combinatory logic. In our initial presentation, we are going to change the notation of subject and formula around to better match the conventions of the combinator definition for now.

2.2.2 K Constant = Opcode 1

The constant combinator K is present in Nock as opcode 1. It takes two arguments, a subject and a formula, and returns the formula unchanged, ignoring the subject. This is useful for building functions that always return a fixed value regardless of the input.

$$*[y\ K\ x] == *[y\ 1\ x] == x$$

2.2.3 S Substitution = Opcode 2

The substitution combinator S is supplied in Nock as opcode 2, the evaluation operator. It takes three arguments, a subject and two formulas, and applies the first formula to the subject, then applies the second formula to the subject, and finally applies the result of the first application to the result of the second application.

```

276
277 * [z S x y] == * [z 2 x y] == * [* [z x] * [z y]]
278

```

2.2.4 The Axiomatic Operators

Because of Nock's cellular nature as a binary tree, we need the ability to access and manipulate nouns. Nouns are binary trees addressed using the natural numbers (in binary, the path down the tree is given by 0 for left and 1 for right).

Classical combinatory logic does not build in the natural numbers, but derives them as Church numerals. Church numerals are a clever way to represent natural numbers as functions. They are more naturally expressed in the lambda calculus, but can be converted from λC to our preferred SKI basis. The Church numeral for zero is $\lambda f. \lambda x. x$, which means that it takes a function f and a value x , and returns x unchanged. The Church numeral for one is $\lambda f. \lambda x. f x$, which means that it takes a function f and a value x , and applies f to x once. The Church numeral for two is $\lambda f. \lambda x. f (f x)$, which means that it takes a function f and a value x , and applies f to x twice, and so forth.

To build Church numerals in SKI, we can use the following translations from λC (presented without derivation from λC to SKI):

```

294 0 = K I
295 1 = I
296 2 = S (S (K S) K) I
297 3 = S (S (K S) K) (S (S (K S) K) I)

```

To cast Church numerals into Nock terms, let's apply our expressions for I, K, and S. (We face our next challenge in translating from SKI to Nock, which is that SKI evaluates from left to right, while Nock evaluates from right ("innermost") to left ("outermost").) We also need to distinguish a Church numeral from the natural number to which it corresponds.

```

303 0 = K I => * [y K I] == * [y 1 [0 1]] == [0 1]
304 1 = * [y I] == * [y 0 1] == / [1 y]
305 2 = * [z S I] == * [z 2 [0 1] [0 1]] == * [* [z [0 1]] * [z [0 1]]]
306 3 = * [z S (S (K S) K) I] == * [z 2 [2 [1 [0 3] [0 2]] [0 1]]]
307   == * [* [z [2 [1 [0 3] [0 2]]]] * [z [0 1]]]
308   == * [* [* [z 2 [1 [0 3] [0 2]]] * [z 1 [0 1]]] * [z [0 1]
309   ]]]
310   == * [* [* [* [z 2 [0 3]] * [z 2 [0 2]]] * [z 1 [0 1]]] * [z [0 1]]]
311   == * [* [* [* [* [z 2 [0 3]] * [z 2 [0 2]]] * [* [z 1 [0 1]]]] * [z [0 1]]]
312   == * [* [* [* [* [* [z 2 [0 3]] * [z 2 [0 2]]] * [* [z 1 [0 1]]]] * [z [0 1]]]

```

A general-purpose increment operator can be built in SKI as well. Let's call this increment operator A (classically written *Succ*) and derive it as follows:

315
316 :: S(S(KS)K)(S(KK)I)
317

318 Very awkward! Fortunately we don't need to continue down this road. We don't
319 actually need Church numerals for Nock because Nock has built-in arithmetic,
320 but they are a useful exercise for understanding how to build more complex com-
321 binators.

322 opcode 3, 4, 5, 10

323 2.3 Idioms & Design Patterns

324 Several of the combinators we examined above are not present in Nock *simpliciter*,
325 but we can build them out of the other pieces. There are also common idioms
326 which are baked in as “compound opcodes” (such as conditional branching, op-
327 code 6).

328 2.3.1 C Flip Operator

329
330 C = S (S (K (S (K S) K)) S) (K K)
331

332 While we can formulate the flip operator C in terms of the Nock translation of
333 S and K, it is more practical to use opcode o. This is almost free due to the binary
334 tree nature of cells.

335
336 *[a C b] == *[[0 3] [0 2]]
337

338 2.4 Nock as Generally Computable

339 While it is beyond our scope to prove the equivalence of the four formulations
340 of general computability, we can briefly demonstrate that Nock does satisfy the
341 requirements of each.

342 SKI and λ C completeness have been established above. Turing machine equiv-
343 alence is established by the fact that we can build a universal Turing machine in
344 Nock. Per (*)Hopcroft1979, p. 148, a universal Turing machine has the following
345 components:

- 346 • A finite, non-empty set of tape alphabet symbols, including a blank symbol.
- 347 • A finite, non-empty set of states, including a start state and one or more
- 348 halting states.

- A transition function that, given the current state and the symbol being read, specifies the next state, the symbol to write, and the direction to move (left or right).

Nock as operator over nouns satisfies these requirements with the natural numbers (including crash `[0 0]` as blank) and cells to constitute nouns, while the transition function is represented by the evaluation rules of Nock itself.

Finally, Nock is generally recursive (or μ -recursive) because it has the necessary building blocks (**Paulsen1995** (**Paulsen1995**); Boolos, Burgess, and Jeffrey (2002), “6.2 Minimization”, pp. 70–71):

- Constant function including zero (opcode 1)
- Increment or successor function (opcode 4)
- Parameter selection or variable access (opcodes 0 and 9 along with opcodes 7 and 8)
- Function composition or program concatenation (opcodes 7 and 8)
- Recursion or looping (opcode 0 with, e.g., opcodes 3, 5, and 6)

Thus Nock can express all μ -recursive functions.

2.5 Virtualization

Nock is a crash-only pure language, which means at the base level it can only succeed and yield a new noun: no side effects, no exceptions, no state. Like a castaway on a desert island, we must build everything we need from raw materials at hand.

Nock has one trick built in and a couple of design patterns to help us get there:

1. Opcode 11 lets us send a hint to the runtime, which can be used to trigger side effects. The Nock expression formally doesn’t know anything about these, but we can use them to print results, read input, and so forth.
2. The runtime can maintain state outside of Nock and supply it as a subject to subsequent Nock expressions. We will do a lot more with this later, but we want to highlight it as a key affordance now.
3. The virtualization pattern means that we can run Nock in Nock.

378 Opcode 11 can encode anything we want: the result of a hint is used by the
379 runtime, and so anything the runtime is willing to work with can be produced.

380 The magic formula at the beating heart of a practical Nock operating system
381 is simply:

382

~~383~~ `*([state -.events] [9 2 10 [6 0 3] 0 2])`

385 That is, apply the Nock formula against a subject pair consisting of the current
386 state and the head of the list of events to process.

387 Chapter 3

388 Interpreter Runtime

389 Abstract

390 Now that we have established a firm foundation for idiomatic programming in
391 Nock, we can build an interpreter using a higher-level language to execute Nock
392 formulas. At this point, we will build the simplest possible Nock interpreter, a
393 tree-walking interpreter. This requires a cursor and some basic memory man-
394 agement but not much more. We will implement some minimalist static hints to
395 permit side effects like output.

396 Bibliography

- 397 Böhm, Corrado and Giuseppe Jacopini (1966). “Flow Diagrams, Turing Machines
398 and Languages with Only Two Formation Rules.” In: *Communications of the*
399 *ACM* 9.5, pp. 366–371. ISSN: 0001-0782. DOI: 10.1145/355592.365646.
- 400 Boolos, George, John Burgess, and Richard Jeffrey (2002). *Computability and*
401 *Logic*. 4th. Cambridge: Cambridge University Press. ISBN: 9780521007580.
- 402 Kleene, Stephen Cole (1952). *Introduction to Metamathematics*. Amsterdam:
403 North-Holland.